

Sprint 2 Progress Report

Evaluation Data Generation on the Fly

Niklas Zoz, Denis Maxheimer, Beyza Ünlü

July 2026

Contents

1	Introduction	2
2	Framework Extensions and Experimental Setup	2
2.1	Tasks and Generation Strategies	2
2.2	Dataset Profiling	2
2.3	Experimental Setup	3
3	Results	4
3.1	Grammatical Error Correction	4
3.2	Spam Detection	6
4	Discussion and Challenges	7
5	Outlook	8

1 Introduction

Sprint 1 established the core architecture of the Generate–Evaluate–Trash (GET) framework and used Grammatical Error Correction (GEC) as an initial proof of concept. The resulting pipeline could generate synthetic benchmark data, evaluate task models and aggregate performance across repeated runs.

Sprint 2 focused on extending this foundation. Spam Detection was added as a second task, support for several generator models was introduced and GEC generation was expanded with an inverse mode guided by the error distribution of the reference benchmark. Dataset profiling was also added so that real and generated benchmarks could be compared at the data level, rather than only through model scores. Where the task allowed it, empirical profiles were also used to guide generation. This report summarises these extensions, the experimental setup and the main findings from the completed GEC and Spam Detection runs.

2 Framework Extensions and Experimental Setup

2.1 Tasks and Generation Strategies

GEC and Spam Detection require different generation procedures. For GEC, the framework now supports forward and inverse corruption. In forward mode, the generator receives a source sentence and chooses which grammatical errors to introduce. In inverse mode, it starts from a known correct sentence and inserts errors according to an empirical distribution obtained from the FCE benchmark using ERRANT. The purpose of the inverse mode is to give the generation process more control over the types and number of errors that appear in the synthetic data.

More generally, inverse generation defines the desired properties of a synthetic example before the example is produced. Rather than allowing the generator to choose a corruption freely, the pipeline first samples a target transformation from a reference profile and then asks the generator to realise it. In the GEC experiments, these target properties were the error types and error counts observed in FCE.

Spam Detection uses class-conditional generation. The local 300-message source file was prepared from the [UCI SMS Spam Collection](#) and is balanced between 150 HAM and 150 SPAM examples. Only the HAM messages are used as generation seeds. A synthetic SPAM example is created by adding one or more profiled spam signals, such as a suspicious link, a monetary reference, urgency, capitalisation, or spam-related keywords. A synthetic HAM example is produced by paraphrasing a legitimate seed while retaining the HAM label. Both synthetic classes therefore originate from HAM messages. Real HAM and SPAM examples remain available for baseline evaluation and profiling.

The generator was separated from the task logic so that models could be changed through the configuration without altering the rest of the pipeline. The completed experiments used MiniMax-M3, GLM-5.2, Tencent HY3, Xiaomi MiMo-V2.5, and DeepSeek-V4-Flash. MiniMax-M3 was tested in both GEC modes; the other GEC generators were tested in inverse mode.

2.2 Dataset Profiling

Sprint 2 added two related forms of profiling. A separate profiling utility provides descriptive statistics for the original datasets. For GEC, this includes sentence and word lengths, character-level similarity between source and correction, correction-complexity categories, frequent words and representative examples. For Spam Detection, it includes class balance, text lengths, frequent words, example messages and rates for rule-based spam signals.

The experimental pipeline also produces a run-level profile that compares real and generated data. GEC pairs are annotated with ERRANT and compared by error-type distribution, errors per sentence, mean edits and the share of supported error categories. Spam profiles compare class balance, signal rates, signal-type distributions and the number of detected signals per SPAM message. Jensen–Shannon divergence is used for the distribution comparisons. The Spam values describe patterns detected by the implemented rules; they are not a direct measure of semantic spam quality.

Table 1 gives a compact example of the information stored in the two run-level profiles.

Table 1: Examples of the task-specific profiling output.

Task	Example profile fields	What the comparison shows
GEC	Error-type distribution; errors per sentence; mean edits; supported-error share	Whether the generated benchmark reproduces the mixture and density of ERRANT-detected errors in the real data
Spam	Class balance; signal rates; signal-type distribution; signals per SPAM message	Whether the generated benchmark reproduces detector-visible class and spam-signal patterns

2.3 Experimental Setup

The GEC experiments used the local FCE benchmark in M2 format, which contains 2,695 sentence blocks. Each run requested 150 synthetic sentence pairs. The Spam experiments used the balanced 300-message SMS dataset described above and requested 150 labelled synthetic messages per run. Every experimental configuration was repeated three times. Generated-data results are reported as mean and standard deviation across these runs, while the real benchmark was evaluated once as a fixed baseline. Source sampling was deterministic, so comparisons with unchanged dataset settings used the same input pool. The requested and usable sample counts were stored separately because rejected, identical, unparsable or otherwise unusable generations could reduce the final number of evaluated samples.

Table 2: Overview of the Sprint 2 experiments.

Task	Source and generation input	Generation setup	Evaluated models and headline metric	Runs
GEC	FCE; 150 requested sentence pairs per run	Forward: MiniMax-M3, MiniMax-M3, GLM-5.2, MiMo-V2.5 Inverse: HY3,	T5 Base Grammar Correction , Grammar Error Correcter V1 , CoEdit-Large ; ERRANT $F_{0.5}$	3
Spam	UCI SMS Spam Collection ; 150 HAM seeds; 150 requested labelled outputs per run	Class-conditional; five generator models across the stored sessions	RoBERTa Base , RoBERTa Spam , SMS Spam Classification with BERT , BERT Tiny ; F1	3

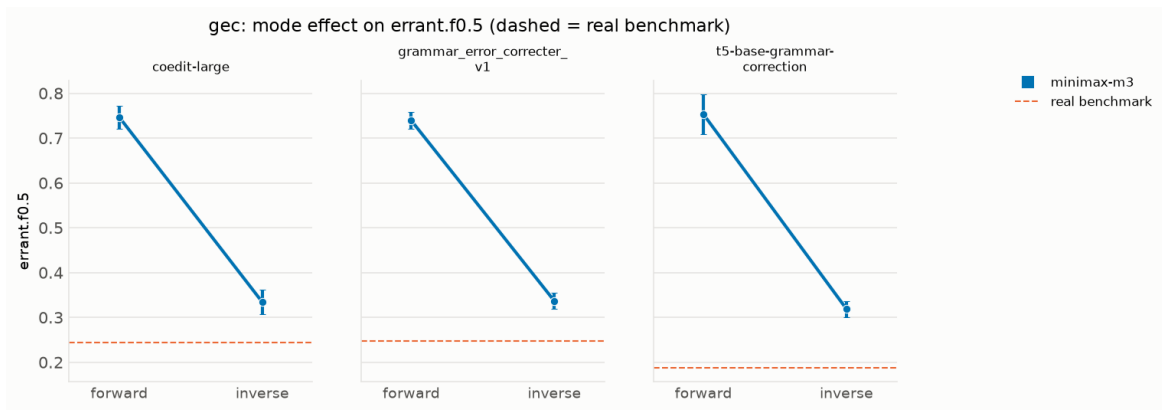
The linked model names in Table 2 point to the Hugging Face model cards used in the completed evaluation. GEC predictions were also assessed with GLEU, ERRANT precision and recall, error-distribution similarity, CoLA, correction extent and number of edits. Spam predictions were

evaluated with accuracy, precision, recall, F1 and false-positive rate, with SPAM treated as the positive class.

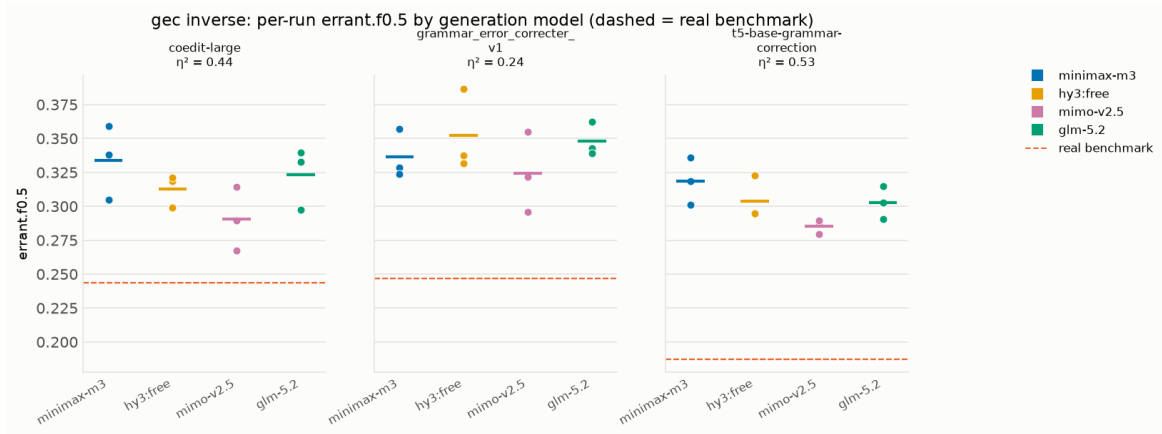
3 Results

3.1 Grammatical Error Correction

The generated GEC benchmarks were easier than the real FCE benchmark in every configuration. The gap was largest with forward MiniMax-M3: the three correction models reached ERRANT $F_{0.5}$ scores between 0.739 and 0.753, compared with 0.187–0.247 on the real data. Using inverse generation with the same generator reduced the scores to 0.319–0.336. The decrease from forward to inverse was 0.403–0.435 across the three evaluated models, which brought the synthetic scores much closer to the real baseline without fully matching it.



(a) Forward and inverse MiniMax-M3.



(b) Generator comparison in inverse mode.

Figure 1: GEC headline results. Dashed lines indicate performance on the real FCE benchmark; points show individual runs or run means as defined in the original plots.

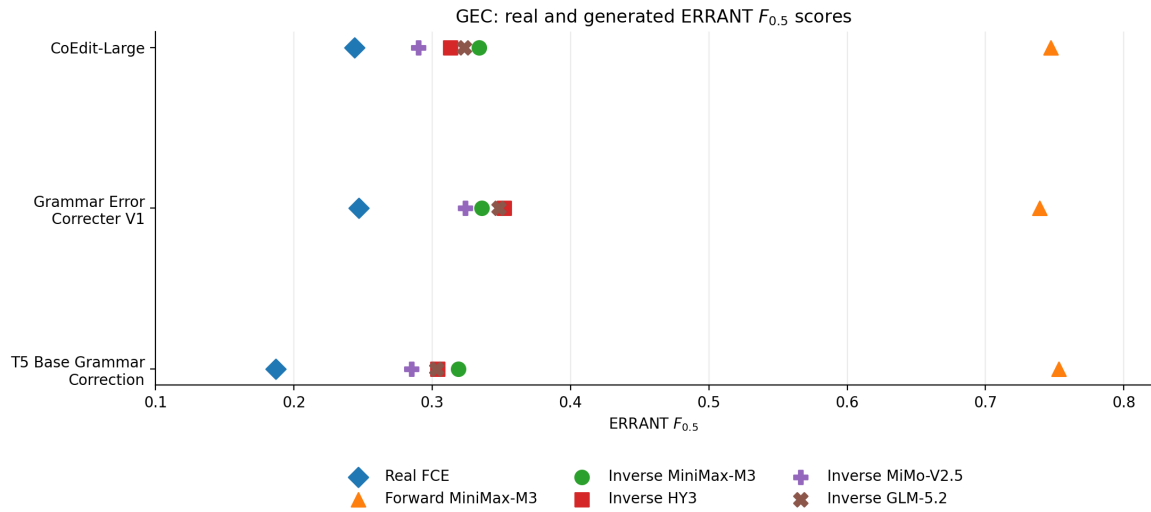
The generation mode also changed the model ranking. Grammar Error Correcter V1 ranked first on the real benchmark, followed by CoEdit-Large and T5 Base Grammar Correction. Forward generation reversed this order, giving a Kendall’s τ_b of -1.0 . Every inverse configuration reproduced the real-benchmark order and reached $\tau_b = 1.0$. This ranking result is based on only three correction models and should therefore be read as evidence for the current experiment rather than a general conclusion.

Differences between inverse generators were smaller than the forward–inverse difference. The

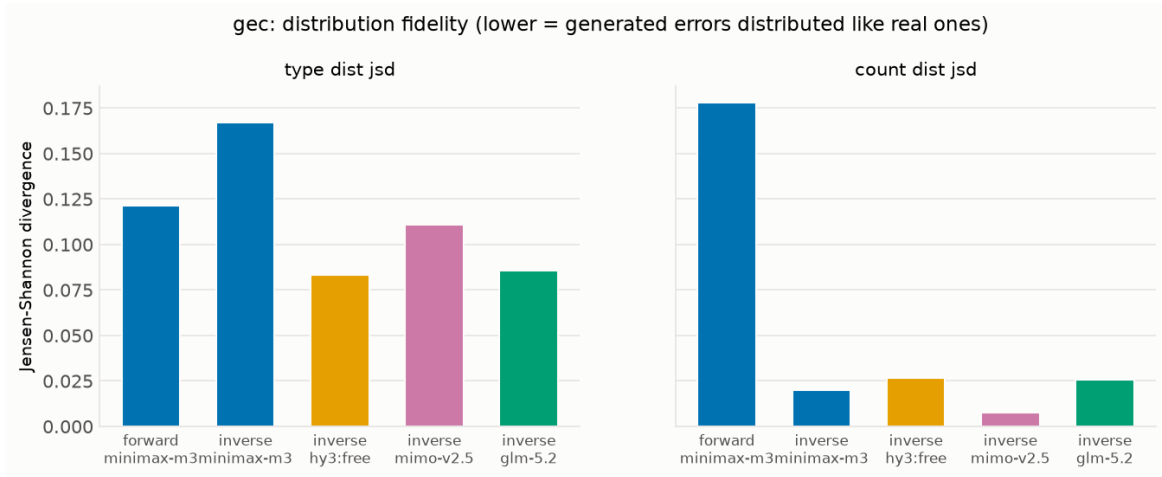
range between the highest and lowest generator-level $F_{0.5}$ score was 0.0435 for CoEdit-Large, 0.0279 for Grammar Error Correcter V1, and 0.0330 for T5. MiMo-V2.5 had the smallest overall mean absolute gap from the real benchmark across the reported metrics (0.082), followed by HY3 (0.0908), GLM-5.2 (0.0949), and MiniMax-M3 (0.104). This does not mean that MiMo-V2.5 was best on every individual metric, but it was the closest overall in the completed comparison.

The fidelity profiles show why score agreement alone is not enough. Inverse generation matched the real distribution of error counts much more closely: count-distribution divergence fell from 0.1780 in the forward experiment to 0.0074–0.0266 across the inverse runs. Error-type fidelity was less consistent. The forward type-distribution divergence was 0.1216; inverse values ranged from 0.0832 to 0.1671 depending on the generator. HY3 and GLM-5.2 improved the type distribution, while inverse MiniMax-M3 was further from the real profile on this measure.

Figure 2 places every completed generated GEC configuration next to the corresponding real FCE score. Panel (a) uses the headline ERRANT $F_{0.5}$ metric, while panel (b) compares the ERRANT-detected error distributions.



(a) Real and generated ERRANT $F_{0.5}$ scores.



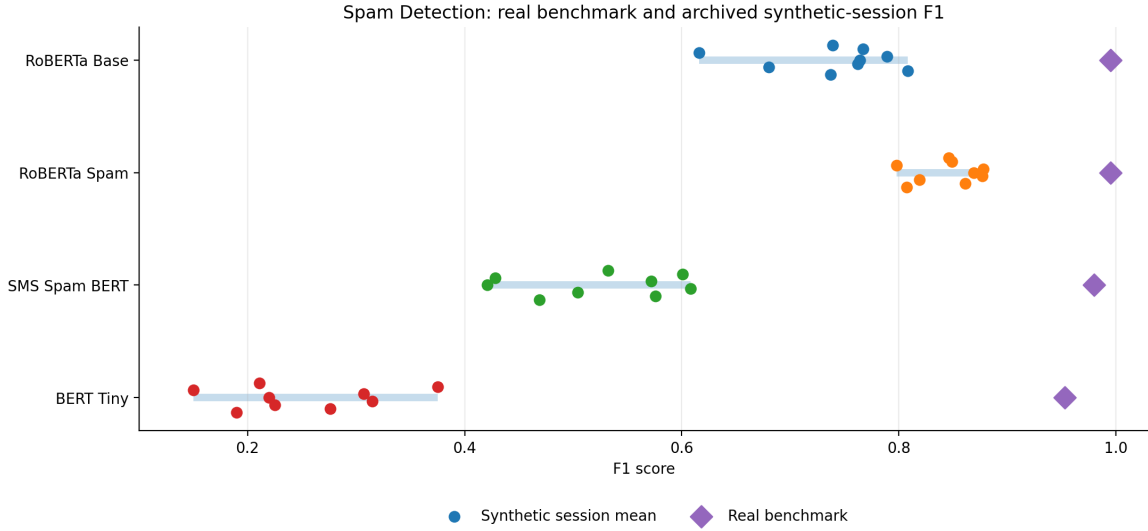
(b) Error-profile fidelity.

Figure 2: Real-versus-generated GEC comparison and error-profile fidelity. In (a), markers show the real FCE baseline and the mean score of each completed synthetic configuration. In (b), lower Jensen–Shannon divergence indicates closer agreement with the real error distributions.

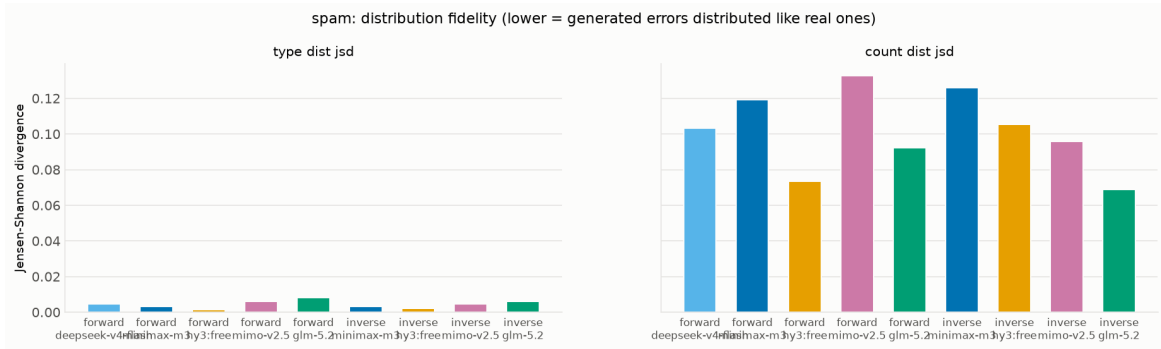
3.2 Spam Detection

All four classifiers performed better on the real SMS benchmark than on the synthetic data. Real F1 scores ranged from 0.953 to 0.995, while generated scores across the recorded sessions ranged from 0.150 to 0.878. RoBERTa Spam was the strongest model on the synthetic benchmarks, with F1 between 0.798 and 0.878. RoBERTa Base followed with 0.616–0.808. The BERT-based models were affected more strongly, particularly BERT Tiny, whose scores ranged from 0.150 to 0.375.

Figure 3 compares the real benchmark F1 score with the mean F1 from every archived synthetic session. Figure 4 separately shows how the results vary across generator models. Panel (b) shows the detector-visible profile differences.



(a) Real benchmark and archived synthetic-session F1 scores.



(b) Detector-visible fidelity.

Figure 3: Real-versus-generated Spam comparison and detector-visible fidelity. In (a), diamonds show the real benchmark and circles show the mean F1 of individual archived synthetic sessions; horizontal lines show the observed synthetic range. In (b), lower Jensen–Shannon divergence indicates closer detector-visible distributions rather than semantic spam quality.

The relative ordering was nevertheless stable. RoBERTa Spam ranked first on every generated benchmark, followed by RoBERTa Base, SMS Spam Classification with BERT, and BERT Tiny. This differs from the stored real order only because the two RoBERTa models exchange first and second place; their real F1 scores both round to 0.995. The resulting Kendall’s τ_b was approximately 0.91 for every recorded Spam configuration.

The generator model had a visible and classifier-dependent effect on the Spam results. MiMo-V2.5 produced the highest recorded scores for the two RoBERTa classifiers, while MiniMax-M3

produced the highest scores for the two BERT-based classifiers. Despite these score differences, the relative ordering of the four classifiers remained stable across the recorded sessions.

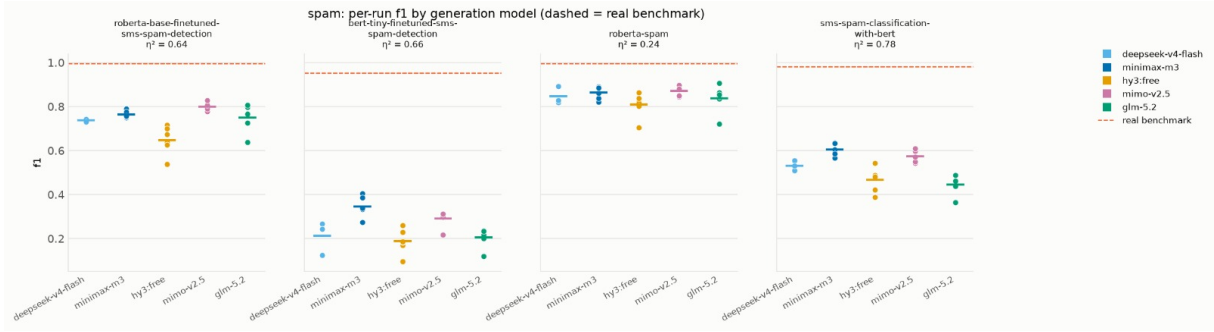


Figure 4: Effect of the generator model on Spam F1 scores across the archived sessions. Points represent individual runs, horizontal markers show the mean, and dashed lines indicate performance on the real benchmark.

4 Discussion and Challenges

The strongest GEC result is the difference between forward and inverse generation. Forward MiniMax-M3 produced a benchmark that was much easier than FCE and reversed the model ranking. Inverse generation reduced the score gap, recovered the real ranking for all four generators and closely matched the number of errors per sentence. It still did not reproduce the real benchmark fully. Synthetic scores remained higher and error-type fidelity depended on the generator. The results therefore support inverse generation as a better starting point for GEC, but not as a finished solution. Profiling-guided generation improved the match with the original benchmark in specific dimensions, especially the GEC error-count distribution but it did not fully reproduce the benchmark’s overall difficulty or error characteristics.

The Spam results are more promising in terms of relative model comparison, even though the synthetic benchmarks were harder than the real SMS data for every classifier. The effect was also uneven: RoBERTa Spam remained comparatively strong, while BERT Tiny lost much more performance. The current experiments show that classifiers respond differently to the generated messages, but they do not identify which individual spam signal causes each change. The mostly preserved ranking is encouraging, although it should not be confused with a match in absolute difficulty.

Across both tasks, benchmark quality cannot be reduced to one number. Score alignment, rank preservation and distributional fidelity can move differently. GEC inverse generation is a clear example: rank preservation and error-count fidelity improved strongly, while error-type fidelity remained generator-dependent. Spam preserved most of the ranking even though the absolute score gap was large. The added profiles are therefore necessary for interpreting the model results rather than treating them as a secondary output.

For GEC, the effect of changing the generation strategy was much larger than the differences among the four inverse generators. In the completed GEC experiments, how generation was constrained mattered more than which of the tested LLMs was used. This finding is limited to the evaluated GEC configurations and does not extend unchanged to Spam Detection, where generator choice still had a noticeable and classifier-dependent effect.

The experiments also exposed practical limitations. Each configuration required three runs of up to 150 generated samples and the current request settings include delays, timeouts and retries. Full comparisons are therefore slow when several generators and task models are included. Requested sample size is not always the same as usable sample size. In the forward MiniMax-M3 GEC

experiment, the three runs contained 149, 147, and 146 usable examples. Possible causes supported by the pipeline and unit tests include refusals, unparsable outputs, unchanged generations and samples rejected by the optional judge; however, the available run artifacts do not identify the exact cause of every missing sample. The pipeline records effective counts and writes results after each completed run, which prevents an interrupted session from losing all previous work.

5 Outlook

The next sprint will continue the implementation of Sentiment Analysis, building on the classification-task structure introduced for Spam Detection. In parallel, the team will explore knowledge-graph-oriented tasks as a possible future direction. The codebase will also be reviewed, debugged and cleaned to improve robustness and prepare the framework for production use.